WELA

How WELA decides

Technical briefing — the rules, the scores and the reasons

Every verdict and every percentage WELA shows, and exactly how it is worked out: how a power cycle is classified and scored, how Storport and disk events are judged, how a fault is tied to the Samsung drive, and how one app, update or service is singled out as the trigger — with the arithmetic for real examples and the questions to expect.

QUESTION 1
How is each power cycle classified — and what does its % mean?

QUESTION 2
How are Storport and disk events judged?

QUESTION 3
When does WELA say the drive caused it — and that it is Samsung's?

QUESTION 4
How is an app, update or service singled out as the trigger?

Version 0.1.0 Branch **main** 2 November 2026 Prepared by Hitesh Kumar

Contents

1 The short version
Principles, the four answers and their four measures

2 From log files to one timeline
Which logs, how they are read, three meanings per event

3 Boots, sessions and time windows
What is read where, and why that long

4 How a power cycle is classified
The decision order, subtypes, restart vs. shutdown

5 The confidence percentage
Formula, weights, why these numbers, worked examples

6 How Storport and disk events are judged
Labels, states, severity, platform vs. drive

7 Did the drive cause it?
Rules, crash codes, effect after boot

8 Which drive — is it Samsung's?
Named, disk number, only drive

9 How apps, updates and services are singled out
Catalogue, window, 1074 decoding, precedence, %

10 Data quality
When the logs themselves are suspect

11 Engineering facts
Stack, privacy, determinism, tests, speed

12 Limits — and what to say
Stated up front

13 Questions to expect
Twenty questions with short answers

A Every number in one place
All settings and weights, with defaults

1 The short version

> WELA reads a PC's Windows event logs and answers four questions for every power cycle with fixed, written-down rules. Every answer shows its evidence, what was expected but not logged, and how strong it is.

Design principles

- **Rules, not a trained model.** The same logs always give the same answer. Every event ID, weight and time window lives in one catalogue file (`wela/catalog.py`) and is pinned by automated tests.
- **Evidence-weighted.** Each event Windows writes when a PC stops carries a weight; the confidence is the sum of what was found, minus penalties for contradictions and log gaps.
- **Redundant signals.** Windows records one stop several times, in different logs and by different components. WELA reads all of them, so one lost event does not change the answer — it only lowers the score.
- **Honest about gaps.** When the logs cannot show something, WELA says *Undetermined* or *Can't tell* rather than guess. Missing records, cleared logs and clock changes are detected and lower confidence.
- **Explainable.** Every verdict lists the events behind it, each one clickable to its raw data.
- **Local only.** Runs on the engineer's PC, serves the UI on 127.0.0.1 and makes no network calls; the logs never leave the machine.

Four answers — four different measures

Question	Answer shown	How strong — the measure	Chapter
How did it end?	Normal shutdown, Restart, Hard shutdown (with a subtype), Sleep / hibernate, Undetermined	**Confidence %** — how complete and consistent the evidence for that ending is	4–5
Did the drive cause it?	Likely, Possibly, No drive evidence, Can't tell — "No (orderly)" for clean stops	**Labelled levels**, no percentage	6–7
Which drive?	Samsung drive, Probably the Samsung drive, Another drive, Drive not identified	**Sure vs. probable**	8

Question	Answer shown	How strong — the measure	Chapter
What triggered it?	e.g. Windows Update, User request, Apps / services hung, App crashed	**Confirmed / Likely / Possibly** with its **own %**	9

Two percentages, not one. In the detail drawer, the large percentage under *Confidence* is about *how the power cycle ended*. The small one next to the trigger — e.g. *App crashed · Possibly · 25%* — is about *what triggered it*. They come from separate weights and answer separate questions; a power cycle can be a 100%-certain hard shutdown with only a 25% software lead.

52
automated tests, including a demo case with known answers

0.2 s
to analyse 19,640 events / 640 power cycles

0
installs — Python standard library and Windows' own wevtutil

4
logs read: System, Application, Storport, Partition (optional)

2 From log files to one timeline

> Every event, from every log, becomes one record on one UTC timeline. Each record is then tagged with what it means.

Which logs, and what each contributes

Log	What WELA takes from it
System	Power-cycle events (12, 13, 41, 109, 1074, 6005, 6006, 6008 …), disk / NTFS / NVMe driver events, service failures, display-driver timeouts, Windows Update
Application	App crashes and hangs, Windows Error Reporting (including blue-screen reports), installer events
Storage-Storport / Operational (+ Health)	Per-request storage problems — timeouts, resets, retries — often with the drive's vendor, product ID and serial
Partition / Diagnostic (optional)	The drive inventory at each boot: model, serial, firmware, disk number — turns "Disk 1" into a named drive

From a folder (e.g. a copy of `C:\Windows\System32\winevt\Logs`, which holds hundreds of logs) only System, Application and Storport are read and the rest are counted as skipped; a file given by itself is always read.

Reading and normalising

- **.evtx** files are read on Windows with the built-in `wevtutil` — Windows renders its own events, so field names and values are exactly what Windows stored. Fallbacks: PowerShell `Get-WinEvent`, then the optional `python-evtx` package on other systems. XML exports are read too.
- Every event becomes one record: time (UTC), log, provider, event ID, record ID, level (1 critical · 2 error · 3 warning · 4 information) and its data fields (named fields, or `param1…n` when Windows does not name them).
- All logs are merged into one timeline sorted by time (System first at equal times, then record ID). **Why UTC:** logs from different time zones or across daylight-saving changes line up; the UI converts to the chosen zone.

Three meanings per event

Meaning	Example	Used for
Power signal	Kernel-Power 41 → "Unclean shutdown reported"	Classification and its confidence (ch. 4–5)

Meaning	Example	Used for
Storage meaning	stornvme 129 → "Reset to device", state *reset*	The drive verdict (ch. 6–8)
Software meaning	Application Hang 1002 → "Application hung", group *app*, kind *hang*, subject *explorer.exe*	The trigger verdict (ch. 9)

Matching is by provider + event ID from the catalogue, so the rules work on meanings and there is one place to change them. A `wela_config.json` next to `run_wela.py` overrides any part of the catalogue without touching code.

3 Boots, sessions and time windows

> Every boot closes the session before it. WELA finds the boots, then asks of each one: how did the session before it end — and what was happening around that moment?

- **A boot** = Kernel-General 12, EventLog 6005 / 6009 and Kernel-Boot 27 within 180 s of each other. Any one of 12 or 6005 is enough (or 27 with BootType 0 / 1), so a boot is found even when a marker is missing. Kernel-Boot 27 BootType 1 = a Fast Startup boot; BootType 2 = a resume from hibernate, which is *not* a boot.
- **A session** = everything between two boots. **The last sign of life** = the last event of any log before the next boot — the anchor for a hard shutdown, because a power loss writes nothing at the moment it happens.
- **Sleeps** inside a session are rebuilt from Kernel-Power 42 (enter) and 107 (resume), Power-Troubleshooter 1 (sleep / wake times and wake source), 506 / 507 (Modern Standby) and Kernel-Boot 27 BootType 2 (hibernate resume). Any one of them is enough to rebuild an episode.



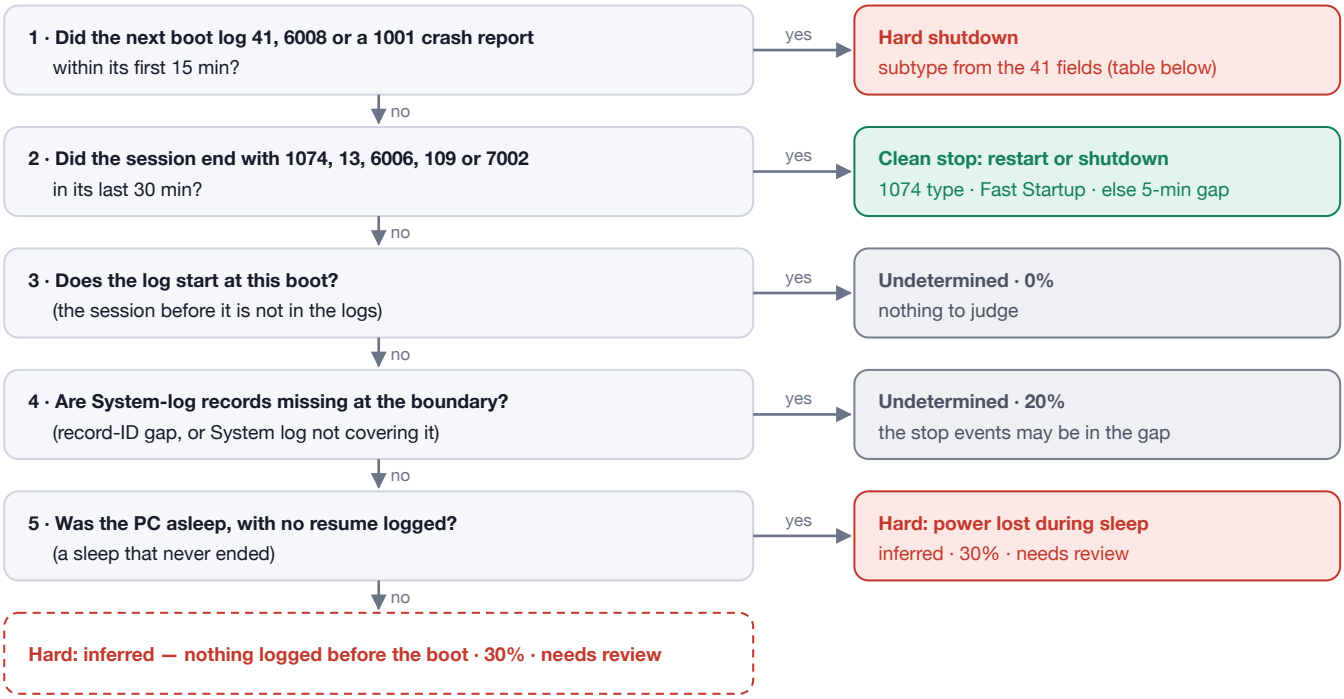
Not to scale. For a sleep the same windows sit around sleep entry and wake. Every length is a setting (appendix A).

Window	Length	Reads	Why this length
Stop signals	last 30 min before the last sign of life	1074, 13, 6006, 109, 7002	An orderly shutdown writes them within seconds to minutes; 30 min tolerates slow shutdowns such as update installs.
Hard signals	first 15 min after the boot	41, 6008, 1001	Windows writes them while booting; the crash report (1001) can follow minutes later, once the dump is processed.
Drive	60 s before the last sign of life (to +5 s) and 60 s after boot	every storage event, every level	A drive fault that brings a PC down shows seconds before it (demo: resets 32 s and 17 s before). A wider window pulls in unrelated noise.

Window	Length	Reads	Why this length
Trigger verdict	15 min before power-off, 10 min after boot — never past the power cycle before or the one after	app, update, service events	Long enough for an install or a hang that builds up; never another power cycle's events.
App / update lists	from the previous boot / wake to this power-off; from this boot to the next power-off	everything, for the engineer	A slow cause can start hours earlier; nothing is hidden. One cycle's "after" list is the next one's "before" list.

4 How a power cycle is classified

> The order of the checks is the design. Windows' own report of an unclean stop beats everything; then a clean stop; only when both are absent does WELA infer — and it says so.



Why hard signals come first: a PC can log a clean stop and still lose power before it finishes. Kernel-Power 41 is written by the kernel at the *next* boot, when it finds the previous stop was not completed — it is Windows' own after-the-fact verdict, so it outranks anything written beforehand. When both appear, the power cycle is a hard shutdown with a conflict penalty (chapter 5).

Hard-shutdown subtype — read from the fields of event 41 (first match)

Evidence	Subtype	What it means
`BugcheckCode` ≠ 0, or a 1001 crash report	Crash (bugcheck 0x…)	Windows stopped with a blue screen; the code says why (chapter 7)
`PowerButtonTimestamp` ≠ 0	Power button long-press	Someone held the power button — usually because the PC had stopped responding
`SleepInProgress` ≠ 0, or a sleep never resumed	Power lost during sleep / hibernate	Battery drained or power cut while asleep
Stop events logged, then 41	Shutdown started but did not complete (hang)	The shutdown hung on the way down

Evidence	Subtype	What it means
None of the above	Power loss / hang / forced reset	Power cut, hard reset or a freeze — nothing in the logs says which

Restart or shutdown?

1. The 1074's shutdown type (param 5), matched in English, German, French, Spanish, Italian, Dutch, Japanese, Chinese, Korean and Russian.
2. The next boot is a Fast Startup boot (Kernel-Boot 27 BootType 1) → **shutdown** (Fast Startup: the kernel hibernated).
3. Otherwise the stop-to-boot gap: under 5 min → **restart**, else **shutdown** — flagged "*inferred from gap*".
4. A 1074 that says restart but a boot more than 30 min later → warning, needs review.

---

5 The confidence percentage

> Confidence = how much of the evidence Windows normally writes for this kind of ending was found, minus penalties for contradictions and log gaps. It is an evidence score on a 0–100% scale — not a statistical probability.

Clean stop	`min(1, Σ clean-stop weights) + 0.10 if the type is stated − penalties`
Hard shutdown	`Σ hard-shutdown weights + 0.30 if no stop signal was logged − penalties`
Nothing logged	`0.30 — an inferred hard shutdown`
Undetermined	`0.20 at a log gap · 0 when the log starts at this boot`
Shown as	`clamp to 0 … 1, then round(confidence × 100) %`

Clean-stop evidence	Weight
User32 1074 — shutdown requested	0.35
Kernel-General 13 — OS stopped	0.30
EventLog 6006 — event log stopped	0.30
Kernel-Power 109 — kernel shutdown initiated	0.20
Winlogon 7002 — user logoff	0.05
Type stated — 1074 says restart / power off, or a Fast Startup boot	+0.10

Hard-shutdown evidence & penalties	Weight
Kernel-Power 41 — unclean shutdown reported	0.50
EventLog 6008 — previous shutdown unexpected	0.40
1001 — bugcheck / crash report	0.20
No stop signal at all before the boot	+0.30
Conflict: 13 + 6006 logged, yet 41 / 6008 on boot	−0.25
System records missing at the boundary	−0.30

Why these numbers

- **By how directly the event records the ending.** 41 is the kernel's own verdict at the next boot (0.50). 6008 comes from a different component — the event log service noticing it was not closed cleanly — so it is independent confirmation (0.40). A crash report confirms a crash, but not every hard stop is a crash (0.20). On the clean side, 1074 is the request itself (0.35); 13 and 6006 are the kernel's and the event log's last clean words (0.30 each); 109 shows the shutdown started, not that it finished (0.20); a logoff also happens without any shutdown (0.05).
- **So that one weak signal never passes review, and two independent ones do.** The review line is 60%: 1074 alone 45% · 13 alone 30% · 1001 alone 50% — *review*; 13 + 6006 = 60% · 6008 alone 70% · 41 alone 80% — *pass* (both hard cases include the +0.30 for a silent end).
- **Capped at 100%:** once the expected evidence is all there, more of it adds no certainty.
- **Hand-set, not fitted.** There is no labelled set of claims to fit them to yet; they were set from how Windows writes these events and checked against the demo cases. They live in `catalog.py`, can be tuned without code, and calibration on 20–30 closed claims is the next step.

Worked examples

ID	Situation	Calculation	Shown
T2	Normal shutdown	0.35 + 0.30 + 0.30 + 0.20 + 0.05 = 1.20 → capped 1.00; + 0.10 stated type → capped 1.00	100%
T6	Shutdown, 6006 lost	0.35 + 0.30 + 0.20 + 0.05 = 0.90; + 0.10 = 1.00 — 6006 listed as "expected, not logged"	100%
T13	Restart, no 1074	0.30 + 0.30 + 0.20 + 0.05 = 0.85; no type stated, so no bonus; restart inferred from a 1-min gap	85%
T5	Power loss	41 0.50 + 6008 0.40 + silent end 0.30 = 1.20 → capped	100%
T7	Crash 0x9F	0.50 + 0.40 + 1001 0.20 + 0.30 = 1.40 → capped	100%
T10	Session just ends, nothing logged	inferred 0.30 + warning	30% · review
T11	37 System records missing at the boundary	undetermined 0.20 + warning	20% · review
—	Clean stop logged, yet 41 + 6008 on boot	0.50 + 0.40 = 0.90 (no silent-end bonus: stops were logged) – 0.25 conflict	65% · warning → review
—	Power loss, records missing at the boundary	1.20 – 0.30 = 0.90 — strong evidence survives the penalty, but the warning still sends it to review	90% · review

"Needs review", colours and sleeps

- **Needs review** = confidence below 60% (`review_threshold`) *or* any warning: a conflict, a record gap, an inferred ending, a restart that took over 30 min, a sleep without resume.
- **Colour of the percentage** in the UI: ≥ 80% green · 60–79% blue · 35–59% amber · below 35% red.
- **Sleeps** score by how many of their three signals were found — 42 enter, 107 resume, Power-Troubleshooter 1 (or 27 BootType 2 for a hibernate): none 30% · one 60% · two 85% · three 100%. Modern Standby: 90% with both 506 and 507, else 50%. A sleep that never resumed: at most 50%. Demo: S2 (entry not logged) 85%, S3 (Power-Troubleshooter only) 60%, S4 (never resumed) 50%.

---

6 How Storport and disk events are judged

> Each storage event gets a label and a state. The worst state in the minute before the last sign of life decides the drive verdict. Informational events are listed with their fields but never counted.

What Storport is. Storport is Windows' storage port driver. It sits between the disk / file-system layers and the drive's own driver (stornvme for NVMe, storahci for SATA). Its Operational log records problems with individual I/O requests — timeouts, resets, retries, latency — and usually names the drive (vendor, product ID, serial), which makes it the best log for tying a fault to a specific drive.

How a state is assigned (first match)

1. **Known provider + event ID** in the catalogue → a fixed label and state (table below).
2. **Any other event in a Storport log** → state from Windows' own severity: Critical or Error → *error*, Warning → *warning*, Information → *info* (listed, never counted).
3. **Any other event from a storage provider** (name contains stornvme, storahci, storport, nvme, iastor or disk) at Critical, Error or Warning level → state from its severity.

Why severity for Storport. Microsoft does not publish a meaning for each Storport Operational event ID, so WELA does not invent one: it uses the severity Windows itself gave the event. Information-level events still appear in the drawer with their key fields (opcode, status, sense data, latency …) so the engineer can read them. Naming the Storport IDs that matter, from real claim logs, is on the roadmap.

States, worst first

State	Events (catalogue)	Read as
missing	disk 157 — disk surprise removed	The drive dropped off the bus
reset	stornvme 129, storahci 129, Intel RST / VMD (iaStorAC, iaStorA, iaStorVD) 129 — reset to device	Windows reset a drive controller that stopped answering
error	disk 7 bad block · 11 controller error · 51 paging I/O error · 52 predicted failure (SMART) · 154 I/O failed; stornvme 11; NTFS 50 delayed write failed · 55 corruption · 98 volume needs repair · 140 failed to flush; volmgr 161; WHEA 18 / 1; Storport errors	An I/O or the data on the drive failed
slow	disk 153 — I/O retried	The drive was slow to answer; the I/O had to be retried
warning	WHEA 17 / 19 / 47 corrected errors; volmgr 46; Storport warnings	Something was wrong but recovered
info	Partition 1006 disk inventory; Storport information	Listed with its fields, never counted

Two refinements

- **Platform vs. drive.** WHEA-Logger events report errors in the CPU, PCIe or memory platform — not necessarily in the drive. They stay in the window, but are left out of the *drive* state, so on their own they can make "Did the drive cause it?" at most *Possibly*, never *Likely*.
- **Readable commands.** NVMe I/O is translated to SCSI commands by stornvme, so Storport events carry SCSI opcodes. WELA decodes them (e.g. 0x2A WRITE(10), 0x35 SYNCHRONIZE CACHE(10), 0x28 READ(10)) so the engineer can see which command failed.

7 Did the drive cause it?

> Asked only of hard shutdowns and undetermined power cycles. A PC that shut itself down in order was not brought down by its drive: those get *No (orderly)*, with a note if drive errors were logged before it.

Verdict	When (first match)
Likely	A *missing*, *reset* or *error* event on the drive in the 60 s before the last sign of life — or a crash code that points at storage (0x7A, 0x77, 0x7B, 0xED, 0xF4)
Possibly	Slow I/O or warnings in that minute, or only platform (WHEA) errors — or a crash code that is sometimes storage (0x24, 0x9F, 0xEF, 0x133, 0x154)
No drive evidence	Logs cover that minute and the drive logged no warning or error
Can't tell	No log covers that minute, or the log starts at this boot

An undetermined power cycle is held back one step: Likely becomes Possibly, No drive evidence becomes Can't tell — the ending itself is uncertain.

Crash code	Name	Points at storage
0x7A	KERNEL_DATA_INPAGE_ERROR	strongly
0x77	KERNEL_STACK_INPAGE_ERROR	strongly
0x7B	INACCESSIBLE_BOOT_DEVICE	strongly
0xED	UNMOUNTABLE_BOOT_VOLUME	strongly
0xF4	CRITICAL_OBJECT_TERMINATION	strongly
0x24	NTFS_FILE_SYSTEM	sometimes
0x9F	DRIVER_POWER_STATE_FAILURE	sometimes
0xEF	CRITICAL_PROCESS_DIED	sometimes
0x133	DPC_WATCHDOG_VIOLATION	sometimes
0x154	UNEXPECTED_STORE_EXCEPTION	sometimes

Why these codes. The "strong" ones mean Windows could not read from or reach its disk (in-page errors, boot device or volume gone) or a critical process was killed — commonly by failed I/O. The "sometimes" ones are often storage-related (a storage driver stuck in a power transition or a DPC) but have other causes too.

Effect on the drive — the 60 s after boot: *Drive missing after boot* (157) · *Errors* (reset / error) · *Warnings* (slow / warning, or platform only) · *Healthy* · *No data*. A drive that is missing or erroring after the boot supports a drive cause.

"No drive evidence" is not "healthy". Windows keeps its logs on the system drive. If that drive froze instantly, its last events were never saved — a failing drive can look quiet. WELA says this on every such verdict and asks for the drive's own SMART and error logs.

Demo examples

ID	Power cycle	Evidence in the minute before	Verdict
T5	Power loss	2 resets to device (the last 17 s before), 5 I/O retries, 3 Storport warnings — on the Samsung drive	Likely
T8	Power-button long-press	3 resets, 2 retries, 3 Storport warnings; a drive event was the last thing logged	Likely
T7	Crash 0x9F	4 corrected PCIe errors (WHEA, platform only) + crash code 0x9F	Possibly
T9	Power lost while asleep	Logs cover the minute; nothing logged	No drive evidence
T11	Undetermined (log gap)	Quiet, but the ending itself is unknown	Can't tell

8 Which drive — is it Samsung's?

> Every storage event is tied to a physical drive where the logs allow it, and WELA says how sure it is: *named* and *disk number* are certain; *only drive* is marked "probably".

Method	How	Shown as
Named	The event carries the drive's identity: Storport VendorId / ProductId / SerialNumber, a Partition model, or a PCI vendor ID (`VEN_xxxx`) in a device path	Samsung drive — sure
Disk number	The event names `\Device\HarddiskN` ; the Partition inventory of *that same boot* says which drive was disk N (numbers can change between boots, e.g. when a USB drive is plugged in)	Samsung drive — sure
Only drive	The event names no drive, but that boot (or the whole log) has only one drive, and the event does not point at another disk number	Probably the Samsung drive
Not identified	None of the above. WHEA platform events are never guessed — their PCIe device can be anything	Drive not identified

- **Samsung is recognised by** the word "Samsung" anywhere in the event, PCI vendor ID `144D`, or a Samsung part number (`MZ…`, e.g. MZVL2512HCJQ) in a model or product field. Generic vendor strings ("NVMe", "ATA", "SCSI") are ignored.
- **Twelve vendors are known** (Samsung, Western Digital / SanDisk, SK hynix, Kioxia / Toshiba, Micron / Crucial, Intel / Solidigm, Seagate, Kingston, Phison, Silicon Motion, ADATA, Lite-On), so another drive is named rather than just "not Samsung". Intel and AMD PCI IDs are left out on purpose: they also name the CPU's own PCIe ports.
- **One drive, two serial formats** (the serial vs. the NVMe EUI-64 some Windows builds report) are merged into one drive.
- The highlighted vendor is a setting (`highlight_vendor`); Samsung is highlighted in teal everywhere — UI, timeline, HTML and Excel.

---

9 How apps, updates and services are singled out

> Four steps: tag the events that matter, read only the right stretch of time, read who asked for the shutdown, then take the strongest explanation first. A request on record beats circumstantial events; a system-process crash beats an ordinary app crash.

Step 1 — which events count

Group	Events
Apps	Application Error 1000 (crash) · Application Hang 1002 (hang) · .NET Runtime 1026 (crash) · Windows Error Reporting 1001, read by its event name: BlueScreen → blue-screen report, APPCRASH / BEX → crash report, AppHangB1 → hang report, LiveKernelEvent → kernel problem report
System processes	A crash of csrss, wininit, winlogon, lsass, smss or services.exe → *System process crashed*: Windows cannot keep running without them
Windows Update	WindowsUpdateClient 43 started · 19 installed · 20 failed · 21 needs a restart · 22 restart scheduled; Servicing 2 installed · 4 needs a restart
Installers	MsiInstaller 1040 / 1042 transaction start / end · 1033–1037, 11707, 11724 done · 11708 failed · 1038 needs a restart · 1005 restarted the PC
Services	Service Control Manager 7031 / 7034 crashed · 7023 / 7024 stopped with an error · 7043 held up shutdown · 7022 hung on start · 7011 timed out. A 7031 whose recovery action is "Restart the computer" → *Windows restarts the PC*
System	Display 4101 driver stopped responding · Resource-Exhaustion 2004 low on memory · Kernel-Power 105 power source changed (on battery / on AC)

Each event also gets its **subject** — the app (e.g. explorer.exe), the update (KB number), the installed product or the service — which is what the reasons name.

Step 2 — the stretch of time

The verdict reads the **15 min before the last sign of life** and the **10 min after the boot** — but never back past the boot or wake that ended the previous power cycle, and never on past the next power-off. If the previous power cycle was only 30 s earlier, it reads those 30 s ("*… in the 30s since T33 boot*"): anything before belongs to that earlier power cycle. The lists under the verdict show everything between the power cycles; problems older than the 15 min are summed up, e.g. "*Earlier, since T7 boot: Crash report: blue screen x1*".

Step 3 — who asked? The 1074 record

User32 1074 is Windows' own record of a shutdown or restart request: **param 1** the process, **3** the reason text, **4** the reason code, **5** the type, **6** a comment, **7** the user.

The process → a category, first matching path fragment wins:

- **Windows Update** — TrustedInstaller, TiWorker, MoUsoCoreWorker, UsoClient, wuauclt, SetupHost …
- **OEM / vendor tool** — Dell (dcu-cli, SupportAssist), Lenovo, HP, Samsung Magician, firmware / BIOS updaters
- **IT management** — SCCM (CcmExec), Intune (OmaDmClient, IntuneManagementExtension, AgentExecutor), Group Policy scripts
- **Installer** — msiexec, *setup*, *install*; **script** — shutdown.exe, PowerShell, cmd
- **System** — wininit, lsass, csrss; **user** — Explorer, RuntimeBroker, Start menu, Winlogon, LogonUI

The reason code → decoded from its bits — the same in every Windows language: bits 16–23 major reason, bits 0–15 minor reason, top bit = planned.

Code	Decodes to
`0x80020003`	planned · Operating System: Upgrade → **Windows Update**
`0x00050006`	unplanned · System: Unstable → **system unstable**
`0x00000000`	unplanned · Other — a typical user shutdown

When process and code disagree, the code wins for updates: "Update and restart" from the Start menu is asked by the shell, but the code says OS servicing → Windows Update. A system process with an "unstable" code or a "terminated unexpectedly" comment → System process crash.

Step 4 — strongest explanation first, and its percentage

Power cycle	Evidence — first match wins	Verdict	Confidence
Any power cycle			
A 1074 was logged	The request itself: who asked, why, for whom	**Confirmed**	0.70; +0.15 if the reason code agrees; +0.15 if update / installer / service events back it
Normal shutdown or restart, no 1074			
	A service crash with "restart the computer", MSI 1005 or Windows Update 22 in the window	**Likely**	0.60
restarts only	An update or install was waiting for a restart (WU 21, MSI 1038, Servicing 4)	**Possibly**	0.35
	None of these	Not logged	—
Hard shutdown or undetermined, no 1074 — in this order			
1	A system process crashed (csrss, lsass, wininit …)	Likely with 0xEF / 0xF4, else Possibly	0.50 (+0.20 with that crash code)
2	A service crashed whose recovery restarts the PC — the restart did not complete	**Likely**	0.60
3	An update or install started in the last hour and had not finished	**Possibly**	0.40
4	Apps or services hung (1002, 7043, 7022, 7011, hang reports)	**Possibly**	0.35
5	Display driver stopped responding (4101)	**Possibly**	0.30
6	An app or service crashed (1000, 1026, crash reports, 7031 / 7034 / 7023 / 7024)	**Possibly**	0.25
	Logs cover the window and none of the above	No software evidence	—

Power cycle	Evidence — first match wins	Verdict	Confidence
	No log covers it, or the ending is undetermined	Can't tell	—
Then, for every verdict that found something			
Symptom check	The drive verdict is Likely / Possibly with real drive events before power-off, and the trigger is a crash (system process or app), hang, driver timeout or unfinished update / install	same, with a note	–0.15

Why this order and these weights

- **1074 (0.70, Confirmed):** Windows' own record of who asked — the only direct evidence. The reason code agreeing and backing events (+0.15 each) take it to 100%.
- **"Restarts the PC" events (0.60, Likely):** the component says it is restarting the PC, but without a 1074 the request itself is not on record — so Likely, never Confirmed.
- **System process crash (0.50, +0.20):** Windows is built to stop when csrss, wininit or lsass die (crash codes 0xEF / 0xF4) — a direct chain; with the matching code it becomes Likely.
- **Unfinished update (0.40):** power lost mid-install is a known risk, but the update may only have been running at the time.
- **Hang (0.35):** a PC that stops responding is often forced off with the power button — it fits the *long-press* subtype.
- **Display driver (0.30):** graphics-driver timeouts often come just before a freeze.
- **App crash (0.25):** an ordinary app crash cannot take Windows down by itself — it shows what was failing at the time. That is why *App crashed* is never more than Possibly.
- **Symptom check (–0.15):** a drive that stalls makes apps hang and crash. When the drive was also failing, the software events are more likely effects than causes, and the reason says so.

Worked examples

ID	Shown	Calculation
T3	**Windows Update** Confirmed · 100%	0.70 (1074 by TrustedInstaller.exe) + 0.15 (reason 0x80020003 = OS upgrade agrees) + 0.15 (WU 43 → 19 → 21 for KB5099001 before it)
T16	**System process crash** Confirmed · 100%	0.70 (1074 by wininit.exe) + 0.15 (0x50006 System: unstable agrees) + 0.15 (lsass.exe crashed 20 s before)
T2	**User request** Confirmed · 85%	0.70 (1074 by RuntimeBroker.exe — the Start menu) + 0.15 (reason 0x0 fits a user shutdown); nothing to back it up
T13	**Software installer** Likely · 60%	MsiInstaller 1005 "restarted the PC" for DEMO Tools 1 min before; no 1074
T15	**Update was installing** Possibly · 40%	WU 43 started KB5099002 and never finished — the last thing logged
T7	**App crashed** Possibly · 25%	demo.exe crashed before a 0x9F crash; the drive verdict found only platform errors, so no symptom deduction
T8	**Apps / services hung** Possibly · 20%	0.35 (explorer.exe and an editor hung) – 0.15 (the Samsung drive was resetting at the same time)
—	**App crashed** Possibly · 10%	0.25 – 0.15: an app crashed and the drive had problems too — the crash is probably a symptom

Shown with every verdict as context, without changing its score: an update waiting for a restart since earlier in the session; the PC running on battery (Kernel-Power 105); a blue-screen report or an update finishing after the boot; services that held up a hung shutdown (7043); older problems since the previous power cycle.

10 Data quality — when the logs themselves are suspect

> Before trusting any verdict, WELA checks the logs. Where they are incomplete it lowers confidence or refuses to guess.

Check	How	Effect
Missing logs	System, Application and Storport are expected	Warning; verdicts say what was not checked (e.g. "only System-log disk events checked")
Unreadable files	Each file is read on its own	Listed with the reason; the rest of the analysis continues
Missing records	Gaps in record IDs, per file and log	Power cycles at a gap → Undetermined (20%) or –0.30 with a warning
Cleared logs	Eventlog 104 / 1102	Warning, with the time and who cleared it
Clock changes	Kernel-General 1 with a jump of 5 min or more, not within 3 min of a boot	Listed with the process that changed the clock
Time going backwards	Record order vs. timestamps, more than 60 s	Warning
Quiet gaps	2 h or more with no events inside a session and no sleep	Info: possibly an unlogged sleep — or an idle PC
Storport coverage	Is there Storport data for each drive window?	A note on each verdict where only System-log disk events were checked

11 Engineering facts

Area	Fact
Stack	Python 3.9+ standard library only (3,207 lines); `.evtx` read with Windows' own wevtutil — nothing to install. The UI is a local web page in plain JavaScript and CSS (1,174 lines), no frameworks, no CDN, works offline.
Privacy	The local server binds to 127.0.0.1 only; no network calls; cases are saved under `./cases` on the engineer's PC. Customer logs never leave the machine.
Determinism	Same logs + same catalogue → same verdicts. No randomness, no model, no internet lookups.
Configurable	Every event ID, weight, window and vendor rule is in `wela/catalog.py`; `wela_config.json` overrides any of it. The drive window, trigger window, restart gap, review line and quiet-gap length are also in the UI's *Analysis settings*.
Outputs	HTML report, Excel workbook (written directly as Office Open XML — no library), CSV, JSON; a command line for batch runs; the full result is saved as one JSON per case.
Tests (52)	Demo scenario with known answers (16 power cycles, 5 sleeps) · 10; power-cycle rules · 13; which drive · 10; software trigger · 16; folder selection · 2; robustness — 30 rounds of up to 400 random, malformed events through the engine and every export · 1.
Speed (analysis step, laptop)	19,640 events / 640 power cycles in 0.21 s · 98,200 / 3,200 in 1.55 s · 196,400 / 6,400 in 4.41 s. Reading `.evtx` with wevtutil adds time in proportion to file size.

12 Limits — and what to say

Limit	What to say
Validated on synthetic logs so far	Every rule is tested against a demo case with known answers. Next: 20–30 closed claims with known outcomes, and lab PCs where each kind of power cycle is caused on purpose (pull the plug, hold the button, force a crash, sleep and wake).
Weights are set by hand	Set from how directly Windows records each event; the closed-claim set will calibrate them. They can be retuned without code.
The logs live on the suspect drive	A drive that froze may never have saved its last events, so "No drive evidence" is not "healthy". Cross-check with the drive's SMART and error logs.
Storport is judged by severity	Its event IDs have no published meanings; WELA uses the severity Windows gave them and shows every field. Naming the important IDs from real logs is next.
"Probably" drive attributions	Unnamed events are only attributed when one drive exists, and are labelled. Adding the Partition log turns more of them into certain matches.
Who asked is not always logged	No 1074 → "Not logged"; restart vs. shutdown then comes from the 5-min gap, flagged as inferred.
Scope	Rules report evidence and patterns; they do not replace failure analysis of the drive. One PC per case; Windows event logs only — no minidumps or SMART import yet.

13 Questions to expect

Does 100% confidence mean it is certain?

No. It means all the evidence Windows normally writes for that kind of ending is there and nothing contradicts it. It is an evidence score, capped at 100%, not a probability. Calibration on closed claims is the next step.

Where do the weights come from?

From how directly each event records the ending: the kernel's own 41 (0.50) above the event log's 6008 (0.40) above a crash report (0.20). They are set so one weak signal stays under the 60% review line and two independent ones pass. They are in the catalogue and can be tuned without code.

How do you know it is right?

52 automated tests pin every rule, including a demo case of 16 power cycles and 5 sleeps with known answers and a random-input test. Validation on 20–30 closed claims and lab tests is next.

Could it wrongly blame our drive?

"Likely" needs a reset, error or missing-drive event on the drive in the minute before power-off, or a crash code that points at storage. Platform (WHEA) errors alone stop at Possibly; orderly shutdowns are never blamed; undetermined ones are held back a step. And it says whose drive it was.

Could it miss a drive fault?

Yes, if the system drive froze before writing its last events. That is why a quiet minute is "No drive evidence", not "healthy", and why every such verdict asks for SMART data.

Why 60 seconds for the drive?

Faults that bring a PC down show up seconds before it (in the demo, 32 s and 17 s). A wider window pulls in unrelated noise. It is a setting and can be widened per analysis.

Why 15 minutes for software?

Long enough for an install or a hang that builds up, short enough to keep a causal link. It never reaches past the previous power cycle; older events are still listed and summed up.

What does "App crashed · Possibly 25%" mean?

An app crashed in that window and nothing stronger was found. 25% is the crash weight: an ordinary app crash cannot take Windows down, it shows what was failing. If the drive was failing too, 0.15 comes off — 10%.

How do you know who shut the PC down?

User32 1074 records the process, the user, a reason code and a comment. The process maps to a category (Windows Update, vendor tool, IT management, user …) and the reason code is decoded from its bits, which are the same in every Windows language.

And if there is no 1074?

Then "Not logged" — unless an installer, update or service says it restarted the PC (Likely) or an update was waiting for a restart (Possibly). Restart vs. shutdown then comes from the gap, flagged.

What is Storport, and how is it judged?

Windows' storage port driver; its log records problems with individual I/O requests and usually names the drive. Its event IDs have no published meanings, so WELA uses Windows' own severity and lists information-level events with their fields without counting them.

How is the Samsung drive identified?

By name, PCI vendor ID 144D or an MZ… part number in the event; by disk number through that boot's Partition inventory; or — marked "probably" — because it was the only drive.

What if the clock or time zone is wrong?

Everything is compared in UTC; the display zone is a choice. Clock changes of 5 min or more are flagged with the process that made them, and time going backwards is flagged.

What if logs were cleared or records are missing?

Both are detected (104 / 1102, record-ID gaps). Power cycles at a gap become Undetermined or lose 30% with a warning, and go to review.

Is customer data safe?

It runs entirely on the engineer's PC, serves the UI on 127.0.0.1, makes no network calls and needs no installs.

Does it scale?

196,400 events and 6,400 power cycles analyse in about 4.4 s on a laptop.

Why rules and not machine learning?

Claim decisions need answers that are explainable and repeatable, and there is no labelled set of claims to train on yet. Rules can be audited line by line. AI fits on top — writing claim summaries from WELA's results — not in place of the rules.

Can the rules change without a developer?

Yes: event IDs, weights, windows and vendor rules are data in the catalogue, overridable with `wela_config.json`; the main windows are also in the UI's settings.

Why "Possibly" so often?

Because the logs rarely prove a software cause; WELA only says "Likely" or "Confirmed" when Windows recorded the request or the component said it restarted the PC.

What does it not do?

No minidump analysis, no SMART import, one PC per case. Those are on the roadmap.

A Every number in one place

> The defaults in `wela/catalog.py`. Any of them can be overridden in `wela_config.json`.

Setting	Default
boot_cluster_s — boot markers within this = one boot	180 s
boot_window_s — read 41 / 6008 / 1001 after boot	900 s
tail_window_s — read stop signals before the last event	1,800 s
restart_gap_s — stop → boot below this = restart	300 s
long_restart_gap_s — restart that took longer → flag	1,800 s
drive_window_s — drive window before / after	60 s
software_window_s — trigger verdict reads before	900 s
software_after_s — … and after boot	600 s
software_list_max — app / update events per list	100
running_lookback_s — update / install still running	3,600 s
pt1_merge_s — Power-Troubleshooter 1 joins a resume	300 s
unexplained_gap_s — quiet gap worth flagging	7,200 s
time_change_flag_s — clock jump worth flagging	300 s
review_threshold — below this = needs review	0.60

Trigger weight	Value
initiator — 1074 names who asked	0.70
reason_agrees — the reason code agrees	+0.15
supporting_events — events back it up	+0.15
restart_logged — "restarts the PC", no 1074	0.60
restart_pending — waiting for a restart	0.35
critical_crash — system process crashed	0.50
critical_bugcheck — … with 0xEF / 0xF4	+0.20
work_in_progress — update / install unfinished	0.40
hang — apps or services hung	0.35
driver — display driver timeout	0.30
crash — an app crashed	0.25
drive_conflict — drive also failing	−0.15

Power-cycle weights and penalties: chapter 5. Sleep scores: 30 / 60 / 85 / 100% by signals found; Modern Standby 90% or 50%; never resumed ≤ 50%.